

The architecture for InterGenOS is highly optimized for the constraints of a 2B model. The focus on **deterministic routing**, **adaptive prompting**, and **grounded synthesis** effectively mitigates the common "noise" and hallucination issues associated with small local LLMs.

Based on the provided review packet and source code, here is an assessment of the architectural soundness, prompt engineering, and safety gaps.

1. Architectural Soundness (Priority Chain P0–P4)

The **Priority Chain** is structurally sound and follows a "fail-fast, cost-optimized" design. By placing deterministic layers (Cache, Identity, P1/P2) before the LLM (P3/P4), you maximize speed and minimize hallucination.

- **The Cache/Compound Conflict (Issue 1):** You've identified a critical race condition where the cache intercepts a compound query.
 - **Recommendation:** Move the **Compound Detection (P0)** to the absolute start of `route()`, before any cache or identity lookups. If a query is compound, it should be split immediately, and the sub-queries should then individually run through the cache/identity logic.
- **P3 Forced Routing:** Forcing diagnostic queries to P3 (Tool Calling) is a necessary guardrail for 2B models that otherwise "fabricate" system states they cannot see. This design decision correctly prioritizes accuracy over latency for system-critical queries.

2. Prompt Engineering (Classify-Then-Compose)

The **Adaptive Prompting** strategy is a significant breakthrough for the 2B tier. Testing shows that always-on rules cause "attention dilution," leading to regressions in unrelated tasks.

- **Strengths:**
 - **Grounded Synthesis:** The "use ONLY the data from the tool output" instruction in `llm.py` is the most robust way to prevent the 2B model from inventing system data.
 - **Conciseness:** Reducing the base prompt to 3 rules plus 1 modifier allows the model to dedicate more "attention budget" to the actual query.
- **Refinement Opportunities:**
 - **Multi-Modifier Stacking:** Currently, the classifier returns a single type. A query like "What's my hostname?" is both identity and diagnostic. Consider allowing the classifier to return a set of modifiers to avoid choosing between "Who am I" and "How do I check state."

3. Safety Gaps and Blind Spots

The safety implementation is robust for its level (keyword triggers + adaptive refusal), but there are areas for improvement.

- **The "Double-Refusal" Trap:** The safety modifier tells the model to refuse "plainly" and "not explain how". However, if the refusal is *too* plain (e.g., "No."), it might fail your "non-empty" or "quality gate" assertions.
- **Shadow Commands:** The `_SAFETY_TRIGGER_WORDS` list is flat. It catches `rm -rf` but may miss encoded commands (Base64) or indirect destructive actions (e.g., writing a malicious systemd service).
 - **Recommendation:** Since you already have a `tool_registry`, the final safety check should happen at the **execution layer**, where the registry validates the command against a strict allow-list, regardless of what the LLM generated.

4. Evaluation and Grader Design (Section 8)

The 13 auto-assertions are a strong "conscience" for the system.

- **Missing Category:** Consider a **"Command Consistency"** assertion. If the LLM generates a tool call for `df -h`, the synthesized response should contain at least one numerical value or path present in the raw tool output. This would mathematically verify the "Use ONLY tool data" rule.
- **Emotional Framing (Issue 2):** Your classifier struggles with frustrated users.
 - **Recommendation:** Add an "Empathetic/Helpful" modifier that doesn't change the tools used but steers the tone. If no keywords match but the sentiment is high-arousal (detected via regex for "!" or caps), inject a "Support" modifier.

Final Verdict on the "2B Ceiling"

Your current plateau (97–102 PASS) is likely the ceiling for **keyword-based routing**. To reach 106+ PASS, you likely need a **Semantic Router** for the classification step itself. A tiny embedding model (like `all-MiniLM-L6-v2`) could replace the keyword sets in `_classify_query_type` to catch the "emotional" or "indirect" phrasings that currently fail.